

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут
Кафедра інформаційної безпеки

ЗВІТ

з дисципліни «Проектування високонавантажених систем»

Лабораторна робота № 1. Веб-лічильник: пам'ять, диск, flame graph

Виконав: Вовкотруб Олександр Віталійович

Група: ФБ-61мн

Перевірив: Родіонов А. М.

Зміст

1	Мета роботи	2
2	Постановка задачі	2
3	Хід виконання	2
3.1	Середовище	2
3.2	Структура програми	3
3.3	Частина I: лічильник у пам'яті	3
3.4	Частина II: лічильник на диску	4
3.5	Генератор навантаження і методика	4
3.6	Частина III: зняття flame graph	5
3.7	Ноутбук аналізу	6
4	Результати	6
4.1	Частина I: лічильник у пам'яті	6
4.2	Частина II: лічильник на диску	6
4.3	Порівняння сховищ	8
4.4	Частина III: flame graph'и	8
5	Додаткове дослідження: підвищення пропускної здатності	13
5.1	Пам'ять: вузьке місце в генераторі навантаження	13
5.2	Диск: групування комітів	14
5.3	Кроки за мотивами One Billion Row Challenge	15
5.4	Куди йде час тепер	17
6	Висновки	18
7	Джерела	18

1 Мета роботи

Виміряти пропускну здатність HTTP-лічильника при різній кількості одночасних клієнтів і з'ясувати, як на неї впливає спосіб зберігання значення: у пам'яті процесу або у файлі з `fsync` після кожного інкременту. Навчитися знімати flame graph сервера під навантаженням і за ним визначати, де витрачається час і що обмежує систему.

2 Постановка задачі

Потрібно реалізувати веб-сервер з двома маршрутами: `GET /inc` збільшує лічильник на 1, `GET /count` повертає його поточне значення. Навантаження створюють 1, 2, 5 і 10 одночасних клієнтів, кожен робить 10 000 запитів `/inc`. Для кожного запуску треба виміряти час і кількість запитів за секунду (RPS) і перевірити, що фінальне значення дорівнює кількості клієнтів, помножених на 10 000, тобто що жоден інкремент не втрачено.

Робота складається з трьох частин.

- Частина I. Лічильник зберігається в пам'яті процесу.
- Частина II. Лічильник зберігається у файлі на диску, і після кожного інкременту дані примусово скидаються на диск (`fsync`). Результати треба порівняти з частиною I.
- Частина III. Для обох варіантів зняти flame graph сервера під навантаженням 10 клієнтів по 10 000 запитів і пояснити, що він показує.

Обрані обмеження реалізації: мова Swift 6 зі строгою перевіркою конкурентності, HTTP-сервер на SwiftNIO без вебфреймворку, власний генератор навантаження.

3 Хід виконання

3.1 Середовище

Сервер і генератор навантаження працюють на одній машині й обмінюються даними через `loorback` (`127.0.0.1`). Диск спільний з хост-системою, тому фонові дискова активність хоста впливає на результати частини II.

Таблиця 3.1. Апаратне і програмне середовище

Параметр	Значення
Процесор	Intel Coffee Lake (інженерний зразок, 2,6 ГГц), 8 ядер, 16 потоків, L3 16 MiB
Пам'ять	62 ГіБ, під час вимірювань вільно близько 24 ГіБ
Диск	NVMe SSD Samsung 970 EVO Plus 1 ТБ, ext4 (noatime), заповнений на 93 %
ОС	Fedora Linux 44, ядро 7.2.5
Компілятор	Swift 6.4, збірка -c release -xswiftc -g
Профілювання	perf 7.2.6, FlameGraph (2023-11-06)

3.2 Структура програми

Код розділено на бібліотеку сховищ і два виконувані файли.

Таблиця 3.2. Модулі програми

Модуль	Призначення
Sources/CounterCore	протокол CounterStore і сховища: у пам'яті, у файлі
Sources/counter-server	HTTP/1.1-сервер на SwiftNIO, маршрутизація запитів
Sources/loadgen	генератор навантаження на AsyncHttpClient
Tests/CounterCoreTests	конкурентні тести сховищ (swift-testing)
scripts/task1.sh	матриця вимірювань 1/2/5/10 клієнтів, 3 повтори
scripts/flamegraph.sh	запис perf і побудова flame graph

Сервер створює `MultiThreadedEventLoopGroup` з 16 `event loop`'ами (за кількістю логічних процесорів) і підтримує `keep-alive`. Кожне з'єднання обробляється як `NIOAsyncChannel` у циклі `for try await` в окремій дочірній задачі. Маршрути: `GET /inc` повертає 204, `GET /count` повертає 200 і значення, службовий `POST /reset` обнуляє лічильник перед кожним запуском, інші запити отримують 404, помилка сховища дає 500.

3.3 Частина I: лічильник у пам'яті

Сховище `InMemoryCounterStore` тримає значення в `Atomic<Int>` з модуля `Synchronization`. Інкремент виконується однією атомарною інструкцією без блокувань, тому одночасні запити не можуть перезаписати один одного.

```
public final class InMemoryCounterStore: CounterStore {
    private let counter = Atomic<Int>(0)
```

```

public fun incrementNow() {
    counter.add(1, ordering: .relaxed)
}

public fun valueNow() -> Int {
    counter.load(ordering: .relaxed)
}
}

```

3.4 Частина II: лічильник на диску

Сховище FileCounterStore зберігає значення як 20 десяткових цифр і символ нового рядка (21 байт) за зміщенням 0. Інкремент під Mutex читає значення (pread), додає 1, записує його (pwrite) і викликає fsync. Блокуючий ввід-вивід виконується в пулі потоків `NIOThreadPool`, щоб не зупиняти event loop'и. Mutex робить послідовність «прочитати, додати, записати, скинути на диск» неподільною.

```

public fun increment() async throws {
    try await threadPool.runIfActive {
        try self.descriptor.withLock { fd in
            let current = try CounterFile.read(fd, path: self.path) ?? 0
            try CounterFile.write(current + 1, to: fd)
        }
    }
}

static fun write(_ value: Int, to fd: Int32) throws {
    try writeRecord(value, to: fd) // pwrite за зміщенням 0
    try sync(fd, name: "fsync") { fsync($0) }
}

```

3.5 Генератор навантаження і методика

Генератор loadgen спершу викликає POST /reset. Кожен клієнт є задачею в `withThrowing-TaskGroup` зі своїм keep-alive з'єднанням. До старту годинника клієнт відкриває з'єднання прогрівочним GET /count, потім усі клієнти чекають на спільному бар'єрі й стартують одночасно. Запити кожного клієнта йдуть послідовно: наступний надсилається лише після відповіді на попередній. Після завершення loadgen читає /count і порівнює його з очікуваним значенням.

```

for _ in 0..<clients {
    group.addTask {
        try await send(.GET, "/count", expecting: .ok) // прогнів
        await gate.arriveAndWait()
        for _ in 0..<requests {
            try await send(.GET, "/inc", expecting: .noContent)
        }
    }
}
await gate.allArrived()
let start = clock.now
await gate.open()
try await group.waitForAll()
return clock.now - start

```

Скрипт `scripts/task1.sh` збирає сервер і генератор у режимі `release` і для кожного сховища тричі проганяє матрицю 1, 2, 5, 10 клієнтів по 10 000 запитів. Час вимірюється лише для фази інкрементів, RPS дорівнює кількості запитів, поділених на цей час. У таблицях наведено медіану трьох повторів і діапазон від мінімуму до максимуму. Сирі дані записано в `reports/data/task1.csv`.

Щоб окремо оцінити вартість дискової операції, скрипт `scripts/fsync-probe.py` 2000 разів виконує той самий ввід-вивід, що й `FileCounterStore` (`pwrite 21` байта і `fsync`), без HTTP.

3.6 Частина III: зняття flame graph

Скрипт `scripts/flamegraph.sh` запускає сервер, підключає `perf record` до його процесу і проганяє `loadgen` з 10 клієнтами по 10 000 запитів. Семпли збираються з частотою 999 Гц зі стеками через DWARF. Потім стеки розгортаються з іменами Swift і будується SVG.

```

perf record -F 999 -g --call-graph dwarf -p "$SERVER_PID" -o perf.data
perf script -i perf.data \
    | swift demangle --simplified \
    | stackcollapse-perf.pl \
    | flamegraph.pl --title "counter-server --store disk" > flame-disk.svg

```

Звичайний flame graph показує лише час на процесорі. Лічильник на диску більшість часу чекає на диск, тому додатково знято off-CPU графік: `perf record -e context-switches -c 1 -g`. Кожен семпл у ньому відповідає моменту, коли потік заснув, разом зі стеком, де це сталося. Графік зважено кількістю засинань, а не їх тривалістю: для тривалості потрібні `tracepoint`'и планувальника, а доступ до `tracefs` у середовищі вимірювань закритий. Для off-CPU графіків використано 10 клієнтів по 1 000 запитів.

3.7 Ноутбук аналізу

Усі медіани, діапазони і рисунки звіту обчислено в ноутбучі `solution.ipynb`, який опубліковано разом зі звітом. Ноутбук читає лише `reports/data/*.csv`, показує проміжні таблиці `pandas` і перевіряє, що обчислені числа збігаються з таблицями звіту. Рисунки ноутбук зберігає в `assets/` тієї ж теки. Розділ «Жива перевірка» запускає `counter-server` і `loadgen` на порту 21421 на зменшеному навантаженні (10 клієнтів по 1 000 запитів для пам'яті, по 100 для диска), щоб показати, що програма працює і рахує точно; офіційні числа беруться лише з CSV. Повне виконання ноутбука займає близько 15 с.

4 Результати

4.1 Частина I: лічильник у пам'яті

Таблиця 4.1. Лічильник у пам'яті, медіана трьох повторів

Клієнти	Запитів	Час, с	RPS, медіана	RPS, мін...макс	Коректно
1	10 000	1,859	5 378	5 360...5 881	3 з 3
2	20 000	1,756	11 392	10 804...11 796	3 з 3
5	50 000	2,120	23 581	22 091...25 518	3 з 3
10	100 000	3,199	31 256	30 215...34 300	3 з 3

У всіх 12 запусках фінальне значення точно дорівнює кількості запитів. Помилки HTTP не було: `loadgen` завершується з помилкою на будь-якій відповіді, крім 204 і 200.

Один клієнт отримує 5 378 RPS, тобто близько 186 мкс на запит. Цей час іде на передачу через `loopback` і обробку HTTP з обох боків: сам інкремент займає 0,2 % процесорного часу сервера (табл. 4.6). Клієнт чекає відповіді перед наступним запитом, тому один клієнт обмежений затримкою, а не ресурсами. Зі збільшенням кількості клієнтів RPS зростає: у 2,1 раза для 2 клієнтів, у 4,4 раза для 5, у 5,8 раза для 10. Від 5 до 10 клієнтів приріст лише в 1,33 раза. Сервер і генератор ділять ті самі 8 фізичних ядер, і процесор насичується.

4.2 Частина II: лічильник на диску

Таблиця 4.2. Лічильник на диску з `fsync`, медіана трьох повторів

Клієнти	Запитів	Час, с	RPS, медіана	RPS, мін...макс	Коректно
1	10 000	85,64	116,8	116,4...168,3	3 з 3
2	20 000	164,09	121,9	118,3...181,3	3 з 3
5	50 000	407,38	122,7	121,3...178,6	3 з 3
10	100 000	818,08	122,2	121,3...122,8	3 з 3

RPS дискового лічильника не залежить від кількості клієнтів, а час зростає пропорційно кількості запитів (від 85 до 818 с). Кожен інкремент тримає Mutex на час `fsync`, тому інкременти виконуються строго послідовно, і пропускну здатність дорівнює одиниці, поділеній на затримку `fsync`.

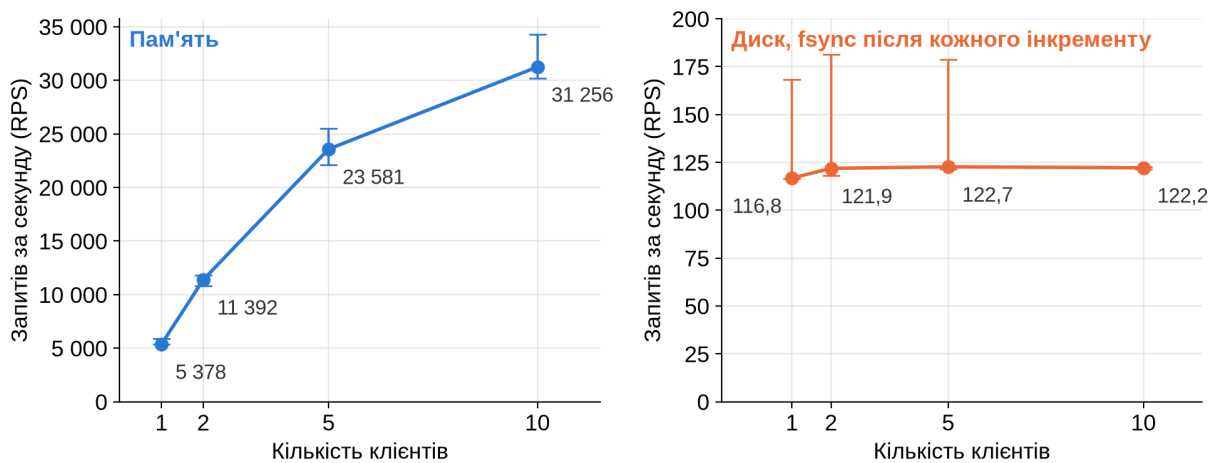


Рисунок 4.1. Медіана RPS і розкид трьох повторів (від мінімуму до максимуму): ліворуч лічильник у пам'яті, праворуч на диску з `fsync`, шкали різні

На рис. 4.1 видно обидві закономірності: крива пам'яті росте й вигинається після 5 клієнтів, а крива диска лишається пласкою на рівні близько 122 RPS. Довгі вуса диска для 1, 2 і 5 клієнтів дав повтор 1, що пройшов у швидкому режимі SSD (табл. 4.3).

Таблиця 4.3. Лічильник на диску, RPS за повторами

Клієнти	Повтор 1	Повтор 2	Повтор 3
1	168,3	116,8	116,4
2	181,3	121,9	118,3
5	178,6	122,7	121,3
10	122,2	122,8	121,3

Повтор 1 для 1, 2 і 5 клієнтів дав 168...181 RPS, а решта запусків 116...123 RPS. Пряма перевірка затримки `rwrite` і `fsync` без HTTP показала те саме перемикання (табл. 4.4).

Таблиця 4.4. Затримка `rwrite` і `fsync` без HTTP, 2000 операцій

	Прогін	Середнє, мс	p50, мс	p99, мс	Операцій/с
1 (після <code>task1.sh</code>)		8,137	8,042	12,343	122,9
2 (через кілька хвилин)		5,480	5,458	5,755	182,5
3 (одразу після 2)		5,472	5,463	5,803	182,7

Затримка `fsync` на цьому SSD має два стабільні рівні: близько 5,5 мс і близько 8,1 мс. Пропускна здатність лічильника точно відповідає їй: 122,9 операції/с без HTTP і близько

122 RPS через HTTP. Накладні витрати HTTP (близько 0,2 мс) на фоні 8 мс `fsync` непомітні. Ймовірні причини двох рівнів: фоновий ввід-вивід хоста (диск спільний і заповнений на 93 %) і внутрішній стан SSD. Ми їх не контролювали, тому медіани в табл. 4.2 відповідають повільному режиму, а швидкий видно в діапазоні.

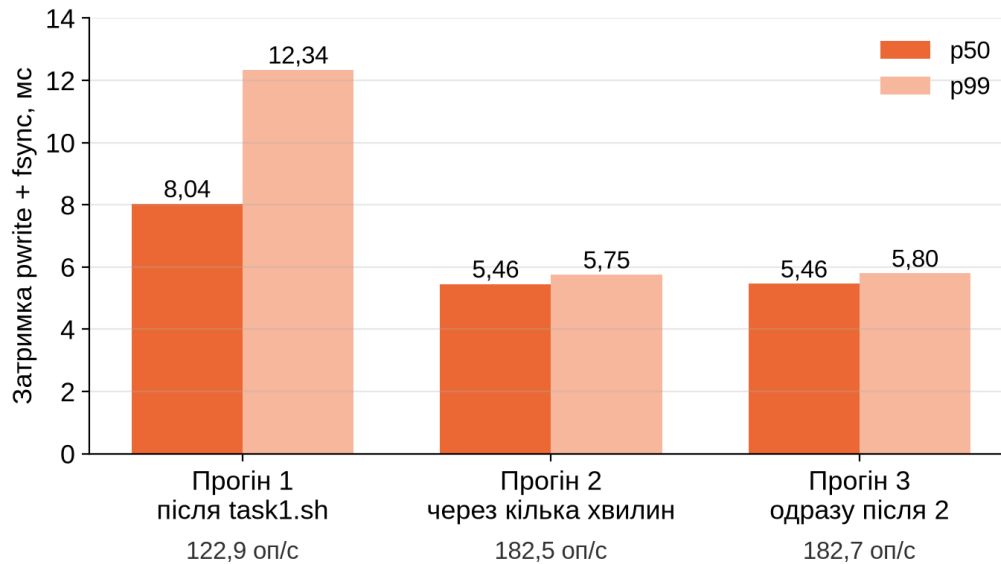


Рисунок 4.2. Затримка `rwrite` і `fsync` без HTTP (p50 і p99, 2000 операцій) і відповідна кількість операцій за секунду

4.3 Порівняння сховищ

Таблиця 4.5. Порівняння пропускної здатності, медіани

Клієнти	Пам'ять, RPS	Диск, RPS	Пам'ять / диск
1	5 378	116,8	46
2	11 392	121,9	93
5	23 581	122,7	192
10	31 256	122,2	256

Лічильник у пам'яті масштабується з кількістю клієнтів, поки не насичується процесор. Дисковий лічильник не масштабується зовсім, і розрив зростає від 46 разів для одного клієнта до 256 разів для десяти. Обидва варіанти коректні: у 24 запусках `task1.sh` і в обох прогонах під `perf` не втрачено жодного інкременту.

4.4 Частина III: flame graph'и

Прогони під `perf`: пам'ять 2,919 с і 34 261 RPS, диск 600,0 с і 166,7 RPS, обидва з фінальним значенням 100 000. Семпли всіх потоків `event loop`'ів зведено під один корінь `NIO-ELT`. Частки нижче пораховано за згорнутими стеками.

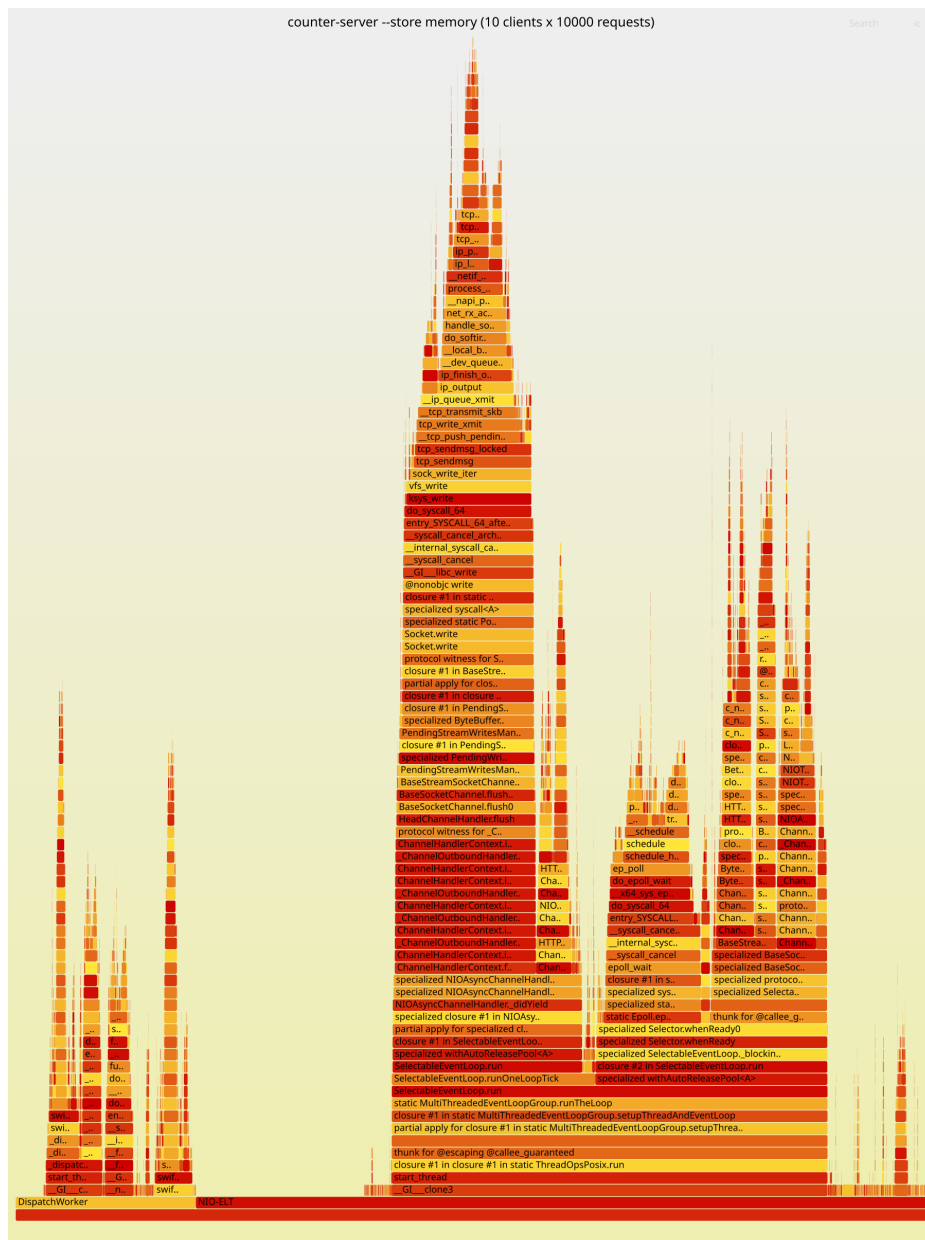


Рисунок 4.3. Flame graph сервера, лічильник у пам'яті, 10 клієнтів по 10 000 запитів

Таблиця 4.6. Розподіл процесорного часу, пам'ять (17 779 семплів)

Що	Частка
Потоки event loop'ів (NIO-ELT), усього	80,2 %
з них запис у сокет (write, TCP-стек ядра)	13,7 %
з них epoll_wait	12,8 %
з них розбір і кодування HTTP	4,7 %
з них читання із сокета	2,0 %
Cooperative pool Swift (Dispatchworker), усього	19,8 %
з них futex (пробудження потоків)	5,9 %
InMemoryCounterStore.increment	0,2 %

На графіку для пам'яті домінують мережевий стек ядра і event loop'и. Найвищий стовпець відповідає запису відповіді в сокет: `Socket.write`, `tcp_sendmsg` і далі весь шлях пакета через `loopback` до прийому. Сам лічильник займає 0,2 %. Ще близько 20 % часу займають потоки `cooperative pool`: обробник з'єднання є `asunc`-задачею, тому кожен запит переходить з `event loop`'а в `cooperative pool` і назад, а пробудження потоків коштує 5,9 % процесора. Отже, межа тут у `HTTP`-сервері і мережевому стеку, а не в лічильнику.

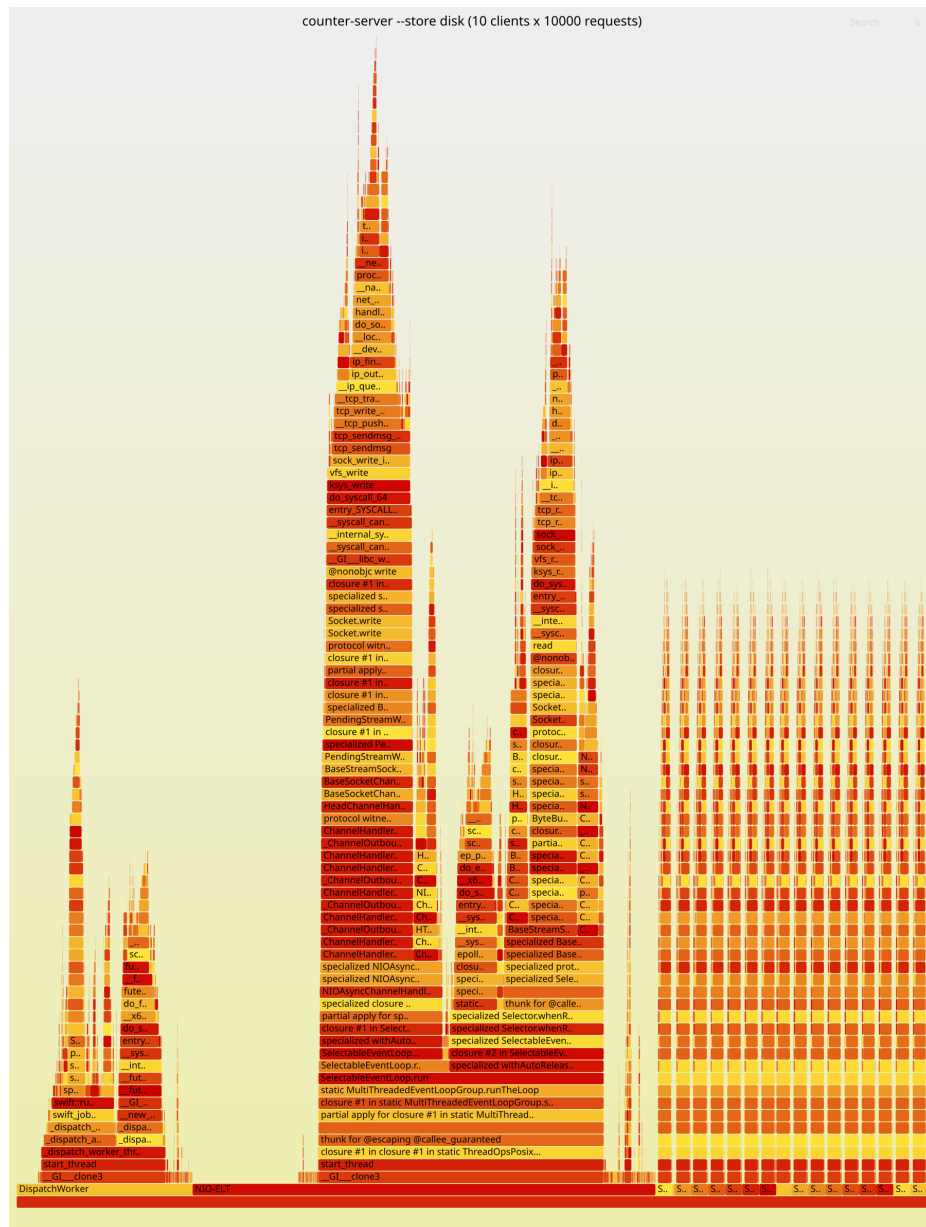


Рисунок 4.4. Flame graph сервера, лічильник на диску, 10 клієнтів по 10 000 запитів

Таблиця 4.7. Розподіл процесорного часу, диск (41 961 семпл за 600 с)

Що	Частка
Потоки event loop'ів (NIO-ELT)	50,8 %
Потоки NIOThreadPool (16 стовпців праворуч), усього	29,9 %
з них fsync (ext4_sync_file, журнал, запис сторінок)	8,5 %
з них pwrite	5,4 %
з них futex (Mutex, пробудження потоків пулу)	4,7 %
з них pread	1,4 %
Cooperative pool (Dispatchworker)	19,3 %

Для диска perf за 600 с зібрав 41 961 семпл, тобто в середньому сервер завантажує близько 0,07 ядра. Для пам'яті за 3 с зібрано 17 779 семплів. Дисковий сервер майже весь час простоює і чекає завершення fsync, а саме очікування на op-CPU графіку не видно: fsync займає лише 8,5 % семплів. 16 однакових стовпців праворуч належать потокам NIOThreadPool: кожен потік прокидається, бере Mutex і знову засинає, бо з файлом одночасно працює лише один з них.

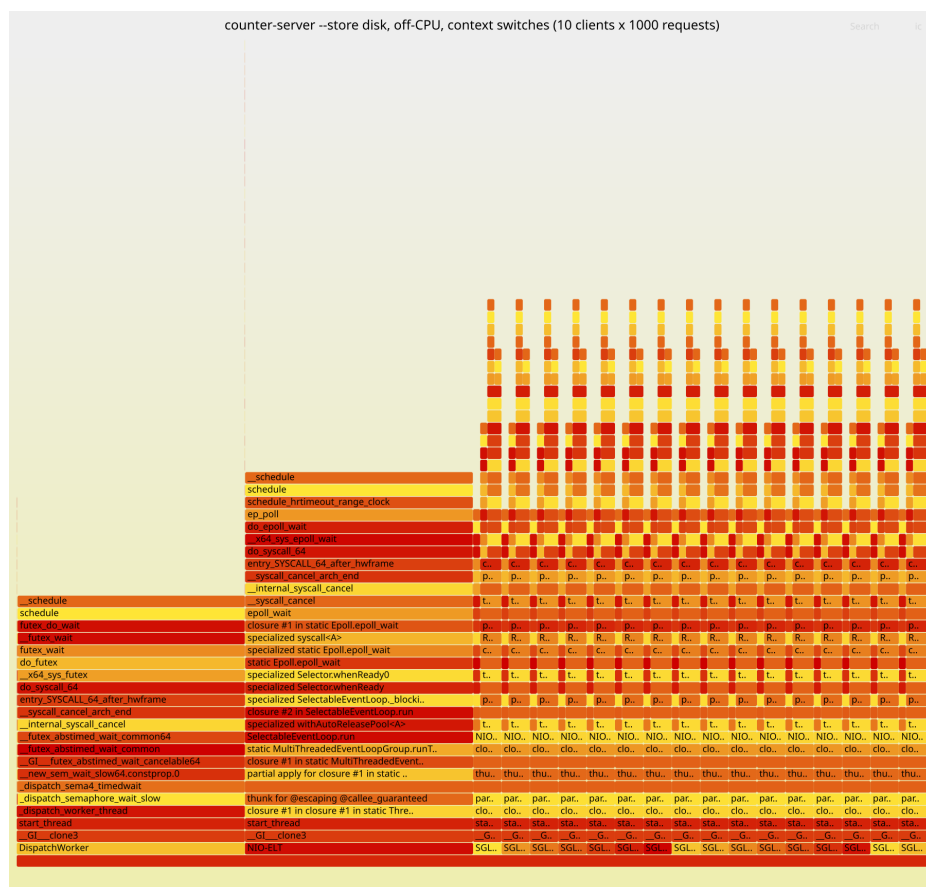


Рисунок 4.5. Off-CPU flame graph, лічильник на диску, 10 клієнтів по 1 000 запитів

Таблиця 4.8. Де засинають потоки, диск (80 287 перемикачів на 10 000 запитів)

Де потік засинає	Частка	На запит
fsync: коміт журналу ext4 (jbd2_log_wait_commit)	12,5 %	1
fsync: запис сторінки (folio_wait_writeback)	12,5 %	1
Mutex у FileCounterStore	12,5 %	1
Потік NIOThreadPool без роботи	12,5 %	1
Cooperative pool (DispatchWorker)	25,0 %	2
Event loop'и в epoll_wait	25,1 %	2

Off-CPU графік показує причину. Кожен інкремент двічі засинає всередині fsync: спершу чекає коміту журналу ext4, потім запису сторінки на диск. Ще один потік пулу на кожен запит стоїть на Mutex, поки інший виконує fsync. Ця черга перед Mutex, за якою стоїть очікування диска, і є вузьким місцем: від 5,5 до 8 мс на fsync, тобто від 120 до 180 інкрементів за секунду незалежно від кількості клієнтів.

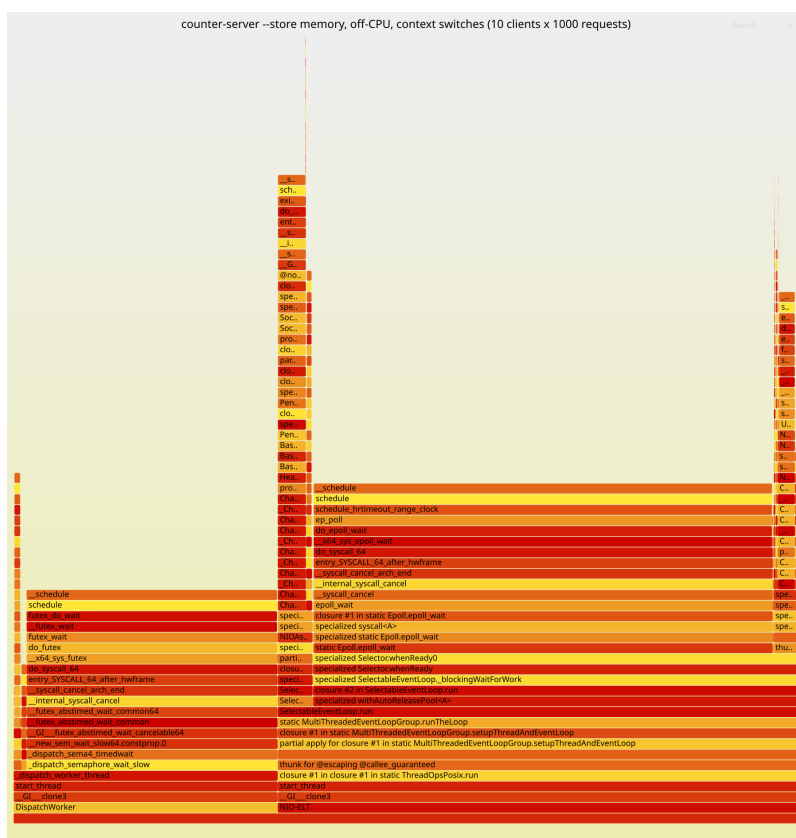


Рисунок 4.6. Off-CPU flame graph, лічильник у пам'яті, 10 клієнтів по 1 000 запитів

Для порівняння, сервер з лічильником у пам'яті засинає 31 351 раз на 10 000 запитів (близько 3 разів на запит): 58,5 % засинань припадає на epoll_wait (event loop'и чекають на мережу), 34,6 % на futex при передачі задач між cooperative pool і event loop'ами. Очікування диска немає, потоки сплять лише в очікуванні наступного запиту.

5 Додаткове дослідження: підвищення пропускної здатності

Після основної частини досліджено, що обмежує кожен варіант і як підняти RPS для сценарію 10 клієнтів по 10 000 запитів, не порушуючи умов завдання: кожен /inc порахований, /count точний, для диска інкремент записано на диск до відповіді, клієнт надсилає наступний запит лише після відповіді. Варіанти завжди вимірювалися почергово (А, В, А, В), щонайменше 3 повтори, у таблицях медіани. Сирі дані: reports/data/throughput-ab.csv і reports/data/1brc-steps.csv. Усі 126 запусків loadgen у цій серії дали точне значення /count.

5.1 Пам'ять: вузьке місце в генераторі навантаження

Незалежний клієнт wrk з 10 з'єднаннями (на кожному один запит у польоті, як у loadgen) показав, що початковий сервер обслуговує близько 72 740 RPS, тоді як loadgen давав лише близько 30 тисяч. Вимір процесорного часу обох процесів за один прогін підтвердив це (табл. 5.1).

Таблиця 5.1. Процесорний час за прогін 10 клієнтів по 10 000 запитів, два повтори

Сервер	Генератор	RPS	CPU сервера, с	CPU генератора, с
початковий	AsyncHttpClient	21 872 / 26 256	11,43 / 11,15	23,55 / 22,82
початковий	NIO	50 786 / 56 547	10,57 / 10,40	4,73 / 4,69
handler	AsyncHttpClient	30 027 / 14 998	5,69 / 5,85	22,64 / 22,81
handler	NIO	94 664 / 83 197	4,82 / 4,92	4,16 / 4,24

AsyncHttpClient витрачає близько 23 с процесорного часу на 100 000 запитів, тобто близько 230 мкс на запит, удвічі більше за весь сервер. Тому зроблено дві зміни. loadgen --engine nio керує кожним з'єднанням через ChannelHandler SwiftNIO і надсилає наступний GET /inc лише після повної відповіді; модель навантаження не змінилась. counter-server --engine handler обробляє запит прямо на event loop'і, без переходу в cooperative pool, який на рис. 4.3 займав близько 20 % процесора.

Таблиця 5.2. Лічильник у пам'яті, 10 клієнтів по 10 000 запитів, 6 запусків на варіант

Сервер	Генератор	RPS, медіана	RPS, мін...макс
початковий	AsyncHttpClient	30 423	4 500...37 185
початковий	NIO	64 696	31 168...76 213
handler	AsyncHttpClient	33 094	26 853...35 117
handler	NIO	98 372	91 712...104 235

Разом обидві зміни дали зростання в 3,2 раза: з 30 423 до 98 372 RPS. Кількість event loop'ів (сітка 4, 8, 16 для сервера і генератора) на результат не вплинула: від 81 до 101 тисячі RPS без закономірності.

5.2 Диск: групування комітів

Проба `scripts/sync-probe.py` порівняла способи скидання 21 байта на диск (3 раунди по 300 операцій).

Таблиця 5.3. Затримка різних способів синхронізації

Метод	Середнє, мс	p50, мс	p99, мс	Операцій/с
<code>pwrite ifsync</code>	5,586	5,436	10,890	179,0
<code>pwrite ifdatasync</code>	1,449	1,411	2,671	690,3
<code>pwrite з O_DSYNC</code>	1,450	1,426	2,232	689,9

`fdatasync` у 3,9 раза швидший: розмір файлу не змінюється, тож оновлення `mtime` не потрібне для читання даних, і `ext4` не комітить журнал. Новий режим `--store disk-group` (`Sources/CounterCore/GroupCommitFileCounterStore.swift`) поєднує це з групуванням комітів, як у WAL СУБД. `/inc` під `Mutex` збільшує значення в пам'яті і ставить свого «чекача» в чергу. Єдиний потік скидання забирає значення разом з усіма чекачами, робить один `pwrite` і один `fdatasync` і лише після цього відповідає всім запитам партії.

```
let batch = state.withLock { state -> (value: Int, waiters: [Waiter])? in
    if state.waiters.isEmpty { state.isFlushing = false; return nil }
    let waiters = state.waiters
    state.waiters.removeAll(keepingCapacity: true)
    return (state.value, waiters)
}
guard let batch else { return }
try CounterFile.writeRecord(batch.value, to: fd)
try CounterFile.sync(fd, name: "fdatasync") { fdatasync($0) }
state.withLock { $0.durable = batch.value }
for waiter in batch.waiters { waiter.succeed() } // відповідь лише після sync
```

Таблиця 5.4. Лічильник на диску, 10 клієнтів по 2 000 запитів, медіани

Варіант	RPS, медіана	RPS, мін...макс
disk (fsync на кожен інкремент)	147,2	114,1...158,6
disk-group, fsync	570,4	559,8...833,0
disk-group, fdatasync	1 737,7	1 661,1...2 042,5
те саме, handler	1 648,1	1 616,9...2 653,7
те саме, handler і генератор NIO	1 733,3	1 664,1...2 945,8

Групування дало зростання в 3,9 раза, `fdatasync` ще в 3,0 раза. Зміни сервера і генератора тут нічого не додають, бо все визначає диск. Без затримки партія містить у

середньому близько 5 з 10 можливих інкрементів. Параметр `--gather-delay-us` змушує потік скидання трохи почекати перед тим, як забрати партію (табл. 5.5).

Таблиця 5.5. Вплив затримки збору партії (handler, генератор NIO)

Затримка, мкс	RPS, медіана	Інкрементів на sync
0	3 405,9	4,99
100	4 396,8	5,76
300	5 438,5	7,42
1 000	3 851,8	9,98

На повному сценарії 10 клієнтів по 10 000 запитів `disk-group` з параметрами за замовчуванням дав 3 269,6 RPS, а із затримкою 300 мкс, handler-сервером і генератором NIO 4 839,6 RPS. В основній частині той самий сценарій давав 122,2 RPS. Значення `/count` в обох прогонах рівно 100 000.

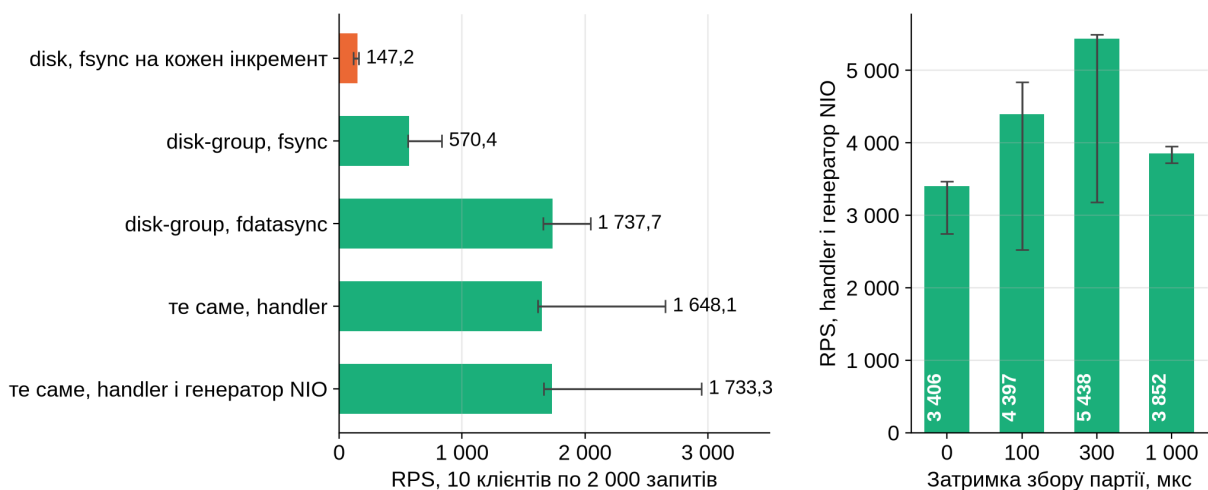


Рисунок 5.1. Лічильник на диску, 10 клієнтів по 2 000 запитів: ліворуч варіанти з табл. 5.4, праворуч вплив затримки збору партії з табл. 5.5; вуса від мінімуму до максимуму

5.3 Кроки за мотивами One Billion Row Challenge

Далі сервер з лічильником у пам'яті оптимізовано кроками, як у One Billion Row Challenge [11...13]: кожен крок спирається на профіль. Скрипт `scripts/bench-engines.sh` запускає `wrk -t10 -c10 -d10s` на `/inc` і одночасно `perf stat` на процесі сервера, лічильники перераховано на один запит.

Таблиця 5.6. Кроки оптимізації сервера, wrk, 10 з'єднань, медіани трьох повторів

Крок	Що змінено	Запитів/с	Тактів на запит	Інструкцій на запит	Промаків переходів
v0	handler: NIOHTTP1, NIOAny, EventLoopFuture	119 914	201 035	78 691	1 072
v1	raw: власний розбір байтів	160 499	114 182	44 344	428
v2	v1 і прапорці компілятора	164 659	106 357	40 361	325
v3	лічильник на кожен потік	не робили			
v4	v1, прив'язка до ядер	169 888	98 419	41 090	361
v5	io_uring, прив'язка до ядер	191 269	65 763	32 005	141

v0 → v1 (+34 %). Профіль v0 показав, що близько половини часу сервера в просторі користувача займає середовище виконання Swift: `swift_retain` і `swift_release` (9 %), перевірки ексклюзивності доступу (близько 6 %), динамічні приведення типів через `NIOAny` (близько 3 %). Рушій `--engine raw` читає `ByteBuffer` напряду і розпізнає маршрут, порівнюючи перші 8 байтів запиту з одним 64-бітним словом. Кінець заголовків шукається векторизованим `memchr`, алокацій немає, відповідь 204 є готовим буфером.

```
static let getInc: UInt64 = 0x636E_692F_2054_4547 // "GET /inc"
static let getCou: UInt64 = 0x756F_632F_2054_4547 // "GET /cou"
static let postRe: UInt64 = 0x6572_2F20_5453_4F50 // "POST /re"

switch word(in: bytes, at: 0) {
case getInc where pathEnds(at: 8):
    return .increment
case getCou where matches(bytes, at: 8, "nt") && pathEnds(at: 10):
    return .count
case postRe where matches(bytes, at: 8, "set") && pathEnds(at: 11):
    return .reset
default:
    return .notFound
}
```

Інструкцій на запит стало на 44 % менше, промаків передбачення переходів на 60 % менше. v1 → v2 (+2,4 %): прапорці `-enforce-exclusivity=unchecked` і `-cross-module-optimization` зменшили процесорний час у просторі користувача ще на 17 %, але час ядра (близько 32 с із 46 с процесорного часу сервера) не змінився. v3 не робили: атомарний інкремент займає 0,28 % семплів профілю, тож розбиття лічильника на потоки дало б щонайбільше стільки.

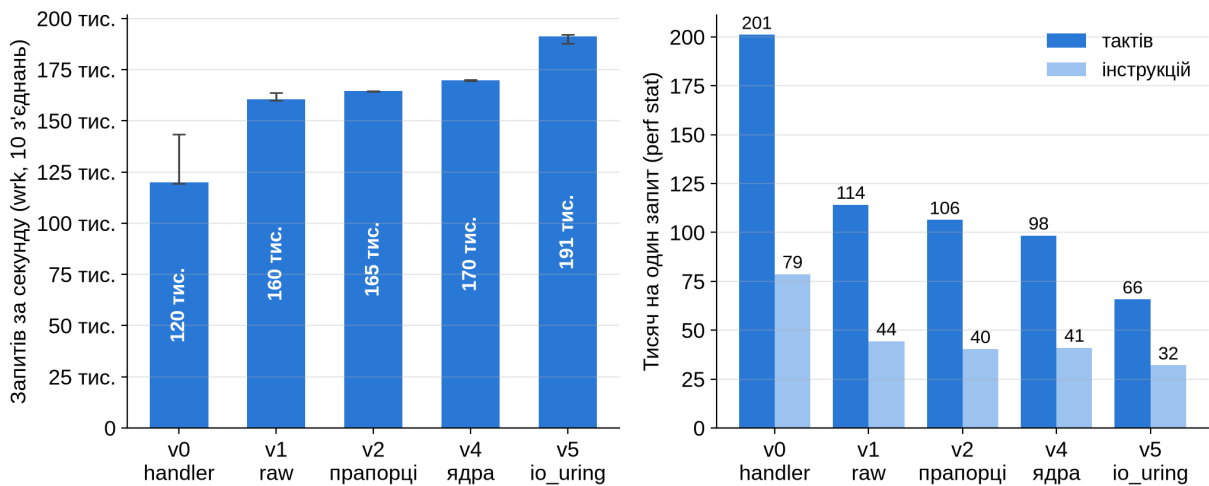


Рисунок 5.2. Кроки оптимізації сервера: ліворуч запитів за секунду (медіана і розкид трьох повторів wrk), праворуч такти й інструкції на один запит за perf stat

v4 (+9,5 %): сервер прив'язано до фізичних ядер 0...3, wrk до ядер 4...7. Перемикань контексту на запит стало на 38 % менше (0,62 замість 1,00), бо прив'язаний event loop частіше застає наступний запит уже в черзі. v5: власний рушій на io_uring (Sources/counter-server/URingServer.swift) подає відповідь і наступний recv зв'язаними операціями і одним системним викликом на раунд замість трьох у циклі epoll. Процесорний час сервера в просторі користувача впав з 16,4 до 2,7 с за 10 с, такти на запит зменшились на 42 %. Режим SQPOLL виявився шкідливим: 16 потоків опитування ядра спляють процесор, і пропускна здатність падає до 123 857 запитів/с, а з прив'язкою до 10 079.

5.4 Куди йде час тепер

Таблиця 5.7. Розподіл часу сервера, v2, perf record

Частина	Частка
Ядро Linux: системні виклики, TCP через loopback, планувальник	60,7 %
Пакетний фільтр ядра (nf_tables, nf_conntrack)	8,5 %
Код сервера і SwiftNIO	17,1 %
Середовище виконання Swift (libswiftCore)	9,7 %
libc	2,5 %

Після v1 і v2 вузьким місцем стала операційна система: близько 69 % часу сервера припадає на ядро. Після io_uring (v5) процесорний час сервера складається приблизно з 27 с часу ядра і 2,7 с простору користувача за 10 с. При 10 з'єднаннях з одним запитом у польоті пропускна здатність дорівнює 10, поділеному на час обороту запиту, а більшу частину цього часу займають два проходи через TCP-стек loopback, системні виклики і пробудження потоків. Далі лишаються лише налаштування хоста (вимкнення conntrack для loopback) або обхід TCP-стека, що вже змінює постановку завдання.

6 Висновки

1. Лічильник у пам'яті на `Atomic<Int>` обслуговує 31 256 RPS (медіана для 10 клієнтів, максимум 34 300) і масштабується з кількістю клієнтів: 5 378 RPS для одного клієнта, у 5,8 рази більше для десяти. Приріст сповільнюється після 5 клієнтів, бо сервер і генератор ділять 8 фізичних ядер. Сам інкремент займає 0,2 % процесорного часу сервера.
2. Лічильник на диску з `fsync` після кожного інкременту дає від 116,8 до 122,7 RPS (медіани) незалежно від кількості клієнтів, у швидкому режимі диска до 181,3 RPS. Пропускна здатність дорівнює одиниці, поділеній на затримку `fsync` (5,5 або 8,1 мс на цьому SSD), бо `Mutex` серіалізує інкременти. Для 10 клієнтів він у 256 разів повільніший за лічильник у пам'яті.
3. Обидва варіанти коректні: у 24 запусках матриці і 2 прогонах під `perf` фінальне значення точно дорівнює кількості запитів.
4. Оп-CPU flame graph для пам'яті показує, що час іде на мережевий стек ядра, `event loop`'и (80,2 %) і перехід у `cooperative pool` (19,8 %). Для диска сервер майже простоює (близько 0,07 ядра), і причину видно лише на off-CPU графіку: два засинання всередині `fsync` (коміт журналу `ext4` і запис сторінки) і черга на `Mutex` на кожен запит.
5. Додаткове дослідження: для пам'яті вузьким місцем спершу був генератор навантаження (230 мкс процесорного часу на запит). Легший генератор і обробка на `event loop`'і дали 98 372 RPS, у 3,2 рази більше. Для диска групування комітів з `fdatasync` дало 3 269,6 RPS на повному сценарії і 4 839,6 RPS із затримкою збору 300 мкс замість 122,2, причому кожна відповідь надсилається лише після скидання на диск.
6. Покрокова оптимізація сервера за профілем підняла стелю з 119 914 до 191 269 запитів/с за `wrk` (власний розбір HTTP, прив'язка до ядер, `io_uring`). Тепер близько 90 % процесорного часу сервера займає ядро, і подальше зростання впирається в TCP-стек `loopback`, а не в код програми.

7 Джерела

1. SwiftNIO: event-driven network application framework [Електронний ресурс]. Apple Inc. Режим доступу: <https://github.com/apple/swift-nio> (дата звернення: 05.10.2026).
2. SE-0410. Low-Level Atomic Operations [Електронний ресурс]. Swift Evolution. Режим доступу: <https://github.com/swiftlang/swift-evolution/blob/main/proposals/0410-atomics.md> (дата звернення: 05.10.2026).
3. AsyncHTTPClient [Електронний ресурс]. Swift Server Workgroup. Режим доступу: <https://github.com/swift-server/async-http-client> (дата звернення: 05.10.2026).
4. Gregg B. Flame Graphs [Електронний ресурс]. Режим доступу: <https://www.brendangregg.com/flamegraphs.html> (дата звернення: 05.10.2026).
5. Gregg B. Off-CPU Analysis [Електронний ресурс]. Режим доступу: <https://www.brendangregg.com/offcpuanalysis.html> (дата звернення: 05.10.2026).

6. perf: Linux profiling with performance counters [Электронный ресурс]. Режим доступа: <https://perfwiki.github.io/main/> (дата звернения: 05.10.2026).
7. fsync(2), fdatasync(2). Linux manual page [Электронный ресурс]. Режим доступа: <https://man7.org/linux/man-pages/man2/fsync.2.html> (дата звернения: 05.10.2026).
8. Jones E. Durability: Linux File APIs [Электронный ресурс]. Режим доступа: <https://www.evanjones.ca/durability-filesystem.html> (дата звернения: 05.10.2026).
9. RocksDB Wiki. Pipelined Write [Электронный ресурс]. Режим доступа: <https://github.com/facebook/rocksdb/wiki/Pipelined-Write> (дата звернения: 05.10.2026).
10. wrk: a HTTP benchmarking tool [Электронный ресурс]. Режим доступа: <https://github.com/wg/wrk> (дата звернения: 05.10.2026).
11. Morling G. The One Billion Row Challenge [Электронный ресурс]. Режим доступа: <https://github.com/gunnarmorling/1brc> (дата звернения: 05.10.2026).
12. Wuerthinger T. 1BRC steps [Электронный ресурс]. Режим доступа: <https://github.com/thomaswue/1brc-steps> (дата звернения: 05.10.2026).
13. Billion Row Challenge step by step [Электронный ресурс]. QuestDB. Режим доступа: <https://questdb.com/blog/billion-row-challenge-step-by-step/> (дата звернения: 05.10.2026).
14. Axboe J. liburing [Электронный ресурс]. Режим доступа: <https://github.com/axboe/liburing> (дата звернения: 05.10.2026).